ChatGPT

基于 DOM 数据的交易辅助分析模块设计方案

背景与目标

交易中的 **DOM**（Depth of Market，即盘口挂单数据）分析旨在利用买卖挂单的动态，提升交易决策的精准度和风险控制能力。传统的K线/价格走势分析提供宏观趋势判断，而DOM数据可以从微观层面补充关键信息。例如，通过查看关键价位的未成交挂单数量变化，可以判断某支撑/压力位是否有真实大单支撑，从而分辨**真实突破**还是**虚假诱多/诱空** [1] [2]。在趋势判断上，K线足够指明大方向，但引入DOM快照有助于在关键价位 **精细化进出场 timing**，尤其是识别 **假突破 vs 真突破** [2]。

同时需要认识到DOM的局限：盘口数据**噪音高、变化快**，每秒快照在流动性高的品种（如纳指期货MNQ）上可能包含大量“钓鱼单”（挂上就撤），必须结合成交量验证信号，不能孤立使用 [3]。此外，**数据量巨大**（每秒快照一小时就是数千条）且主要对 **短周期** 有效（1-5分钟内的微观博弈），对于数小时的波段趋势参考价值有限 [4]。因此，本模块的设计重点是**滤除噪音**，提取对交易有用的特征，并与宏观K线结构相结合，在关键时刻提供**风险提示和决策建议**。

目标：构建一个基于DOM数据的盘中监测模块，集**订单簿失衡、订单流失衡**及**大单压托（墙单）行为**的检测于一体，用于**过滤假突破、确认趋势延续、动态调整仓位杠杆** [5]。模块将与现有交易系统集成，包含后端高速数据处理和前端可视化界面，并支持事件通知和策略自动交易功能。最终希望将盘口线索转化为**可执行的风险控制与仓位决策** [6]。

系统架构

整体架构如下所示：

- **数据源**：通过 IB (Interactive Brokers) API 订阅**Level-2 市场深度**和**逐笔成交**信息 [7]。IB Gateway/TWS 提供的 `reqMktDepth` 和 `reqTickByTickData` 接口将源源不断发送盘口和成交更新。
- **数据采集**：后端启动异步任务接收 IB API 数据，按高频（~10–20Hz）构建**DOM快照** [8]。即每秒多次合并盘口变化和最新成交，形成当前买一到买K、卖一到卖K的价量快照以及最近成交变化。
- **数据缓冲**：将快照写入**内存环形缓冲区**（NumPy实现），以支持毫秒级读写和固定内存占用 [8] [9]。缓冲区大小窗口N可以覆盖若干秒至几分钟的高频数据，用于后续特征计算。
- **特征计算**：对每条新快照，实时计算一系列DOM指标，如订单簿不平衡(OBI)、订单流不平衡(OFI/CVD)、墙单强度、撤单速率、吸收指数等 [10] [11]。这些特征提炼盘口变化的统计特征和异常行为。
- **行为识别**：基于特征，进一步识别特定的盘口行为事件，例如**压单/托单**（大单持续挂出）、**拉单/撤单**（大单瞬时撤销）、**吸收**（大单吸筹/出货）、**扫单**（快速吃单跨档），以及**假突破**迹象评分等 [10] [11]。这一层形成对盘口动态的语义解释。
- **风控/仓位集成**：将DOM分析结果输送给**风险控制和仓位管理引擎**。例如输出趋势强度评分、假突破概率、墙单状态等信号，用于动态调整仓位大小、过滤不利信号、触发风控措施 [12] [13]。
- **事件总线**：模块将重要事件和信号发布到事件总线，供系统的通知模块、自动策略等订阅 [14]。同时，通过 WebSocket 推送给前端界面，确保用户实时监控。
- **前端展示**：Web 前端界面（使用纯 HTML/JS/CSS）通过 WebSocket 接收DOM数据与事件，并在**监控面板**中可视化 [14]。用户可在界面上查看实时指标、信号强度，可交互查看事件细节，并根据建议手动或自动执行交易决策。

该架构确保DOM数据从采集、处理到应用和展示形成闭环，实时、高效且对交易决策具有参考价值。

数据采集与存储

数据订阅：使用 IB API 提供的 **Level 2 市场深度**（`reqMktDepth`）获取盘口挂单数据，以及 **逐笔成交**（`reqTickByTickData`）获取即时成交明细 [7]。为了覆盖主要盘口行为，深度订阅档位数K可以配置（如20档），实际返回由市场可见深度决定。逐笔数据订阅 `Last` 和 `BidAsk` 类型，以捕捉每一笔成交及报价变动。采样频率统一提升至 **10~20次/秒**，即每50-100毫秒构造一次盘口+成交快照 [15]。前端显示可进一步下采样至 ~5次/秒以平衡性能与可读性。

内存缓冲结构：为了高效存取高频DOM数据，采用 **NumPy 环形缓冲区** 来存储最近若干窗口的快照 [16]。示例设计：窗口大小 `WINDOW=N`（如10000帧，相当于几分钟的数据），最大深度 `MAX_LEVELS=30`（满足K<=30的情况），买卖两侧 `SIDES=2`，记录字段如价格和挂单量 `FIELDS=2`。则缓冲区数组维度 `[N, 2, 30, 2]` [17]。辅助数组记录每帧实际深度层数、盘口中位价mid、时间戳等 [18]。插入采用索引循环覆盖，保证内存占用固定且支持快速随机访问最近的数据。

```
WINDOW, MAX_LEVELS = 10000, 30
book = np.zeros((WINDOW, 2, MAX_LEVELS, 2), dtype=np.int32)  # [:,0,:,:]买盘, [:,1,:,:]卖盘; fields: [price, size]
valid_levels = np.zeros((WINDOW, 2), dtype=np.int16)         # 每帧实际深度层数（买/卖）
mid_tick = np.zeros(WINDOW, dtype=np.int32)                  # 中位价tick
ts_ns = np.zeros(WINDOW, dtype=np.int64)                     # 时间戳（ns）
```

每次新数据到来时，将当前买卖档位数据写入下一位置，并记录实际档数和中位价等。计算指标时，可直接对缓冲区指定窗口范围切片以利用向量化运算。例如计算 `OBI_k` 时，会取当前帧买卖前k档累积量，然后计算比值 [19]。

半持久化存储：为了支持**一周内的历史DOM数据回溯**，设计半持久化的数据保存方案。考虑到每秒1帧、一周约60万帧数据，使用高效的列式存储（如 **Parquet**）或轻量级时序数据库来保存数据 [19]。实现方案：
- **批量写入**：将实时DOM快照以批量方式写入本地文件（例如每分钟或每1000帧写一次）。使用Parquet可以方便按时间范围查询。也可采用内存映射文件或SQLite等轻量DB。
- **保留与清理**：只保留最近7天的数据文件，定期删除过旧的数据（或启动时清理）。
- **快速查询**：提供后端API按**合约(symbol)** 和 **时间范围** 查询DOM历史数据，返回该期间的时间序列数据。接口如 `GET /api/dom?symbol=MNQ&start=2025-09-10T00:00&end=2025-09-10T01:00`，可返回JSON数组或供下载的CSV。 [20] 为了快速响应查询，可以在写入时对数据按时间分区索引，查询时直接定位文件块。
- **数据内容**：历史数据既可以是**原始盘口快照**（大数据量，供精细回放），也可以是**预计算的指标序列**（小数据量，供快速分析）。考虑到性能，默认提供指标序列（如每秒的OBI、OFI、壁垒强度等），必要时再按需加载原始深度数据。

通过以上机制，实现DOM数据“一次写入，多次读取”，满足实时分析的同时，也支持事后复盘和验证模型。

特征提取与算法原理

本模块提炼了盘口与订单流的**六大核心特征**，用于量化市场微观结构的不平衡和异常行为：

- **订单簿不平衡（OBI，Order Book Imbalance）**：衡量买卖挂单力量对比。计算方法是取前N档买单总量和卖单总量的比值，例如 [21]：

$$\text{imbalance} = \frac{\sum_{i=1}^{N} \text{bid}i^{\text{size}}}{\sum}^{N} \text{bidi^{\text{size}} + \sum.$$ }^{N} \text{ask}_i^{\text{size}}

当imbalance > 0.60时表示买盘相对更厚（偏多），< 0.40则卖盘更厚（偏空） [22] 。该指标反映盘口静态供需强弱对比。

- **订单流不平衡**：分为**累计成交量差（CVD）**和**订单流差（OFI）**两部分。
- CVD（Cumulative Volume Delta）：在滚动窗口W秒内，主动买量减去主动卖量的累积和 [23] 。公式：$CVD_t = \sum_{k=t-W+1}^{t} (\text{buy_vol}_k - \text{sell_vol}_k)$，反映短期内主动资金流向。
- OFI（Order Flow Imbalance）：刻画盘口挂单变化对价格的推动 [24] 。简化公式例如：

  $$OFI_t = \Delta \text{best_bid_sz}t \cdot \mathbf{1}}} \;-\; \Delta \text{best_ask_szt \cdot \mathbf{1},$$ }

  即如果中位价上涨，则取买一挂量增加量，若中位价下跌则取卖一挂量减少量，相减得到净值 [24] 。这个OFI度量买卖盘挂单变动的净效应，可视为无需逐笔成交也能推测的订单流强度。
  - 我们对CVD和OFI都计算**z-score标准化**以衡量其显著性 [25] 。窗口W内，z值超过 +1表示买方流显著占优，低于 -1表示卖方显著占优。

- **堆单/撤单强度**：捕捉盘口挂单的剧烈变化。定义每秒相对于上一秒的深度总量变化 [26] ：

- $\Delta \text{bid_depth} = \sum_{i=1}^N \text{bid}i^{size}(t) - \sum(t-1)$ （买盘深度变化） }^N \text{bid}_i^{size
- $\Delta \text{ask_depth} = \sum_{i=1}^N \text{ask}i^{size}(t) - \sum(t-1)$ （卖盘深度变化）}^N \text{ask}_i^{size
  然后取窗口W内这些Δ值的z-score：
- 当 Δ为正时，算作**堆单**：如 $z_stack_bid = zscore(\max(\Delta \text{bid_depth},0))$ 表示买盘挂单突然增加的显著性；类似 z_stack_ask 表示卖盘挂单突然增加 [27] 。
- 当 Δ为负（挂单减少）时，算作**撤单**：如 $z_pull_bid = zscore(\max(-\Delta \text{bid_depth}, 0))$ 表示买盘挂单突然撤单的显著性； z_pull_ask 类似 [27] 。

- 高的 z_stack 表明瞬间出现了异常大的新增挂单（压单堆积），高的 z_pull 则表明出现了撤单潮。

- **墙单强度**：识别盘口中的巨大挂单（通常称为“墙”）。方法是在每个快照中检查**最优1-3档**是否存在远高于平均水平的大单：例如超过近期平均值 + 若干标准差。若连续多帧都存在类似巨量挂单，则认为一侧出现了强支撑/阻力墙。可以定义一个墙单强度指标，综合考虑该大单的量级（相对于正常挂单均值的z值）和持续时间。墙单强度高表示在该价格附近有**持续的厚单**托盘或压盘 [28] 。

- **撤单速率**：结合上述墙单和撤单指标。当价格朝某方向运动接近大墙单价位时，如果观察到挂单量突然骤降且几乎没有成交配合，这表明可能是大单故意撤出（**诱导**行为） [29] 。撤单速率可以用 z_pull 在墙单价附近的峰值来衡量——例如当卖盘存在大墙却突然在价格接近时被撤单，可判定为卖盘撤单速率高。这通常意味着先前的挂单可能是虚假流动性（spoofing）。

- **吸收指数**：衡量某价位上挂单对主动成交的吸收程度（Iceberg行为）。典型情境：某关键价位上有大量挂卖单，买盘不断以该价成交但价格**迟迟上不去**，同时挂卖数量迅速补回 [30] 。这意味着大卖单在持续吸收买盘。判断标准：在近W秒内，**主动买成交量显著高**（超过均值+1σ），**价格几乎未上涨**（中位价变动 <1-2个tick） [30] ，且观察到卖一反复出现挂单被吃又补（z_pull_ask 和 z_stack_ask 交替出现） [31] 。满足这些即可标记**吸收事件**（如记录 `absorb_up=True` 表示上方有吸收卖压） [32] 。类似地，下跌方向有 `absorb_down` 。

- **扫单检测**：指短时间内连续的大额市价单把盘口多档吃掉的情形。比如在2秒内卖盘成交激增，买盘挂单总量快速骤减，同时中位价连续跨档下跌≥2个tick，这就是一次**向下扫单(sweep_down)** [33] 。可定

义事件：若 `sell_vol` 在2秒内 > 平均+1.5σ 且 $z_pull_bid > +1.5$，并且价格快速下跌超过一定幅度，则触发 `sweep_down=True` [32]。同理 `sweep_up` 用于检测快速拉升吃单的情况（买单猛烈吃掉挂卖盘）。

- **假突破启发式**：通过上述特征组合侦测**可能的假突破**情形。启发式逻辑包括 [34]：

- **撤墙+无跟随**：在突破关键价位时，如果原本的阻力墙突然撤单（卖盘厚度骤降）却没有相应的成交量涌现（CVD/OFI未见大幅跟随），这是假突破的信号。
- **OBI/OFI 快速反转**：突破发生后一侧挂单优势和订单流在极短时间内迅速逆转（如原本偏多的OBI瞬间转弱），表明突破缺乏真实支撑。
- **价差变化**：真突破往往伴随交易活跃、买卖价差可能**缩小**或保持正常；若突破时反而价差扩大（流动性变差），则值得怀疑。 综合这些迹象可计算一个**假突破概率 (fake_breakout_prob)** ∈ [0,1]，当多项条件同时出现时该概率升高 [35]。例如满足撤单+成交不足等条件时赋值0.7以上。

阈值设置：初始建议使用经验阈值，如 N=5档、W=20秒窗，imbalance判断>0.6或<0.4为偏多/偏空，z-score绝对值>1算显著信号，>1.5算强信号 [36]。具体事件如 `sweep_down` 定义为2秒内sell_vol大于均值+1.5σ且z_pull_bid>1.5并伴随价格显著下跌 [37]；`absorb_up` 定义为5-10秒内buy_vol高于+1σ且卖盘挂单连续补回，同时价格上涨不足2 tick [37]。这些参数可在实践中微调。需要注意统计特征只提供线索，必须辅以成交量/价格变动确认 [38] ——因此本模块在计算指标的同时，也监测实际成交量变化，避免孤立依赖盘口挂单数据。

行为检测与信号判定

借助上述特征指标，本模块在每个时间点评估交易信号和事件，主要输出包括：

- **事件标记**：对满足条件的盘口行为打上布尔标记。例如：
- `absorb_up` / `absorb_down`：检测到上方/下方吸收行为 [30]。
- `sweep_up` / `sweep_down`：检测到向上/向下扫单事件 [32]。
- 另外可标记 `stacking_buy/sell` 或 `pulling_buy/sell` 来指示当前主要是挂单增加还是撤单（这些可通过z_stack/z_pull符号和大小判断）。在设计上，我们也定义 `wall_state` 枚举来描述盘口墙单状况（如 none、stacking_buy、pulling_sell、absorption 等） [35]。
- **趋势评分** (`trend_score`)：连续型指标，范围 [-1,1]，表示盘口力量对价格趋势的支持程度 [35]。它可综合近期OBI、OFI和价格走势方向计算。例如，可将imbalance映射到 [-1,1]（卖强为-1，买强为+1），再根据OFI/CVD的方向加权确认。如果买盘占优且主动成交也偏多，则trend_score接近+1；反之接近-1；若指标矛盾则在0附近。这提供一个实时**多空偏好**度量。
- **假突破概率** (`fake_breakout_prob`)：如前述，通过检测撤单、成交不充分等模式估计当前价位突破的可信度 [35]。值越高表示越怀疑当前向上突破可能是诱多（或向下突破是诱空）。这一概率可用于风险控制（如减少仓位或不追单）。

- **交易信号判定**：结合DOM指标和价格**结构位**（支撑/阻力）判断明确的做多/做空信号。原则上**只有在预先定义的关键价区**才考虑触发信号，避免区间中部频繁误触 [39] [40]。信号规则如下：

- **做多信号（在支撑区）**：当价格跌至某支撑区域附近，若出现多个看多盘口迹象，则产生做多入场建议。触发条件需满足以下任意3条 [41]：

    1. **位置条件**：中间价 `mid` 落在支撑带范围（允许误差1–2个tick内） [42]。
    2. **挂单确认**：盘口imbalance偏多（>0.60）**且** 买盘堆积显著（`z_stack_bid > +1`） [42]。表示此支撑有大量买单挂出。
    3. **成交确认**：订单流也偏多，`z_ofi > +1` **或** `z_cvd > +1` [42]。表示有主动买盘跟进，不只是被动挂单。

4. **吸筹/价差**：出现吸收买盘迹象（`absorb_up = True`），**或** 价差并未扩大（保持在2个tick以内） [43]。价差小通常意味着流动性良好，交易顺畅。
5. **价新低停止**：在最近W秒内价格没有再创新低（LL未更新） [44]。表明下跌动能可能衰竭。
   如果以上5条中至少3条成立，则判定**看多信号成立** [41]。模块会生成一个做多建议，包括入场价区、信心水平等。

附加：建议的风控措施会伴随信号给出，如**止损**放在结构位下方约8–12个tick [45]；**目标位**可以设为前期中枢或前高（约10–20 tick利润）作为第一目标，支撑区上沿（20–40 tick）作为第二目标 [46]。当达到第一目标且部分盈利时，建议将止损上移至开仓价上几tick（保本止损） [46]。

- **做空信号（在压力区）**：价格涨至阻力区域，如出现多个看空迹象则提示做空。判定条件与做多对称，需满足以下任意3条 [47]：

1. **位置条件**：`mid` 落在阻力带范围（±1–2 tick） [48]。
2. **挂单确认**：盘口imbalance偏空（<0.40）**且** 卖盘堆积显著（`z_stack_ask > +1`） [48]。说明阻力位有大量卖单压制。
3. **成交确认**：订单流偏空，`z_ofi < -1` **或** `z_cvd < -1` [48]。有主动卖盘配合。
4. **扫单/价差**：出现向下扫单事件（`sweep_down = True`），**或** 突破时卖方并未缩小价差（spread仍≥2 tick，没有因为买盘涌入而变紧） [49]。
5. **价新高停止**：最近W秒内价格不再创新高（HH未更新） [49]。
   满足≥3条则判定**看空信号** [50]。附加止损建议放在阻力带上方8–12 tick [51]，目标和移动止损策略与做多类似镜像处理 [51]。

- **硬性规则**： [52]

1. 永不在价格区间中部下单——即只有接近明确支撑或阻力区域，才根据DOM信号操作，避免在无意义区域交易 [53]。
2. **特殊时段调整**：在日内开盘前后2–3分钟或重大消息发布等剧烈波动时段，提高警惕。这些时候DOM数据失真严重，建议将所有阈值翻倍严格（降低信号敏感度）或直接不交易观望 [54]。

上述规则实现了价格结构和DOM信号的“共振”触发：只有当K线结构位和盘口信号同时满足一定条件才行动 [2]，从而减少假信号。同时模块提供的信号**强度**（例如满足条件的数量或z值大小）也可用于评估信号置信度。对于人工交易者，这些规则以**建议**形式呈现；对于自动策略，则可编码为触发条件。

风控与策略集成

DOM分析模块不仅提供信号建议，也直接融入仓位和风控管理，以动态适应市场微观变化：

- **动态仓位缩放**：根据DOM给出的趋势和风险信号调整每笔交易的仓位大小 [35]。实现思路：
- 计算基础风险仓位：每笔交易基础风险金额 $R_0 = \text{账户权益} \times r$（$r$为账户允许单笔风险比例），然后按策略预设的止损距离计算基础张数 $N_0 = \left\lfloor \frac{R_0}{(\text{合约价值/点} \times \text{止损点数})} \right\rfloor$ [55]。这N_0是不考虑盘口的常规仓位。
- 根据DOM信号确定缩放因子f：
  - 若 `trend_score` 明显为负（即盘口反趋势）或者 `fake_breakout_prob > 0.6`（怀疑是假突破），则设 **$f = 0.25$**（仓位减至25%） [55]。
  - 若信号中性无明显偏向，则 **$f = 0.5$**（仓位减半）作为保守起见。
  - 若趋势信号与预期方向强烈一致且未检测到假突破迹象，则 **$f = 1.0$**（全仓执行） [55]。

- 实际下单手数 $N = \max(1, \lfloor N_0 \times f \rfloor)$，至少为1手 [56]。通过这样的动态调整，在不利或存疑时显著降低仓位，在有利时才投入正常仓位，从而控制风险曝露。

- **事件过滤**：利用DOM事件信号来过滤或延迟不利的交易机会 [35] 。例如：

- 如果检测到 `pulling_*` 类型事件（盘口同方向的大单突然撤单）恰好发生在准备做多/做空方向上，则**延迟入场**约30–60秒 [57] 。这避免追入可能的诱骗行情——如看到突破就下单，但主力撤单可能是假突破信号，等待片刻确认。

- 如果检测到 **吸收**(`absorption`)且成交跟进不足的情况，则对已有仓位给出减风险建议：**减半仓位**或**收紧止盈** [57] 。例如做多过程中，发现上方有吸收卖盘，则提示减仓或将止盈目标提前锁定利润。
  通过事件过滤机制，模块在策略临界点上为交易加上“二次确认”，大幅降低陷入陷阱（如假突破失败）的概率。

- **断路器**：设定基于DOM策略的风控断路器，防止连续损失 [58] 。具体规则：如果本策略当天累计亏损达到≥3笔基础风险（$3 R_0$），**或** 连续亏损交易数达到≥3笔，则触发**停止交易**信号 [58] 。一旦触发，模块将暂停后续自动交易信号执行，直到人工重置或下个交易日。这一断路机制保证在策略失灵或极端行情下及时收手，保护账户安全。需要注意，DOM模块作为风险控制的一部分，只能让策略**更保守**（从紧风控），而不应绕过已有的风险限制 [58] 。

- **策略自动化交易**：本模块可作为一个**内部自动交易策略**运行，利用其信号自动下单和风控。集成方式如下：

- 将DOM信号判断逻辑封装为策略引擎的一部分。当满足做多/做空条件时，策略模块自动生成下单指令，包括方向、多空、数量、价位等。下单时遵循上文**动态仓位**计算结果来确定数量，并设置好止损位和初步止盈目标等。所有下单尽量使用**限价单或被动单**进入 [59] ，以减少滑点；遇到 `sweep` 等紧急反向信号或者止损条件满足时，可使用市价单迅速平仓 [59] 。
- 该策略会被纳入系统的**策略面板**，作为**内部策略**出现。例如命名为「DOM辅助策略」，默认加载但初始可设置为不启用（由用户在界面上打开自动交易开关）。作为内部策略，它将有自己的策略概览、订单和盈亏统计页面 [60] 。用户可以在策略列表中看到它，并在Summary标签查看其描述和设置，在Orders & PnL标签查看由其发出的订单执行记录及盈亏 [61] 。
- **订单标识**：所有由DOM策略发起的订单应标记其来源，以便在成交列表中清楚显示。例如在订单备注或ID中加入策略名称，前端也可在订单表格的“来源”列显示策略名称而不仅是“策略”泛称。这样一来，用户复盘时可识别哪些交易由DOM模块自动执行。
- **策略设置**：在策略属性中增加一个布尔开关“启用DOM自动交易”。当关闭时，模块仅提示信号和推荐操作，不会真正下单（纯辅助模式）；当开启时，满足条件即自动下单。建议默认关闭自动交易功能，待用户观察模块可靠性后再手动开启。
- **盈亏评估**：开发专门的PnL汇总评估页面或直接利用策略面板现有功能来评估DOM策略的表现。统计其总盈亏、胜率、夏普比率等。如果已有“Visual Metrics”图表，可复用以展示策略权益曲线等 [62] 。另外，对此策略的每笔交易结果进行标记，可汇总比如“因假突破信号而延迟的交易次数避免了多少损失”之类，以验证模块有效性。
- **特殊策略处理**：注意该策略的特殊性——它依赖实时流数据和其他策略画的结构位等外部信息（假设关键支撑阻力由人工或其他模块提供）。我们需要确保当这些外部条件缺失时不贸然行动。同时，遵守断路器和全局风控：例如停用状态下不应误触发交易。
- **策略执行器**：如果现有框架有策略执行器和事件订阅机制，应该重构/完善使其能良好支持这种高频内部策略。如果没有，则考虑新建一个独立的异步循环监测DOM信号，当满足时通过订单管理模块下单。无论哪种实现，都要保证交易指令执行的可靠性和及时性，并处理好与其他策略或人工单的冲突（例如检查当前是否已有仓位等）。

通过上述集成，DOM模块不只是一个被动分析工具，更成为策略决策的一环，帮助**动态控制仓位风险**并**自动捕捉短线机会**。

前端界面设计

本模块需要在前端提供直观的可视化和交互，使交易者能方便地监控DOM指标和信号，并对自动策略进行控制。我们沿用现有的技术框架，即**纯 HTML + CSS + JavaScript** 实现前端（无需引入新的大型框架），样式上与当前界面一致（可能使用Ant Design风格或自定义CSS），后端通过 **FastAPI** 提供数据服务和WebSocket [63]。

1. “Position & Risk Controls” 面板集成：在仓位与风控控制面板中，新增一个 **DOM 标签页**（放置在现有 S Rules 等标签旁）。该标签页用于展示DOM相关的计算结果和配置选项：

- **DOM指标展示**：以时间序列或数值列表的形式，显示主要DOM指标最近的变化趋势。例如，可以包含一个小型折线图用于OBI/OFI随时间的变化，以及当前时刻的关键指标值。也可用图表+表格结合方式：图表显示过去几分钟OBI、OFI的曲线， [64] 表格或数值块显示当前的墙单强度、撤单速率、吸收指数、fake_breakout_prob 等值。其中多头信号相关的指标用绿色，高亮表示强烈偏多；空头信号指标用红色。若有事件发生（如当前帧被标记了absorb或sweep事件），可以在此页显著标注或列出最近事件。
- **DOM Settings 按钮**：在DOM页的标题栏增加“⚙ DOM设置”按钮。点击弹出 **DOM参数设置** 模态窗口。窗口内容包括可配置的所有DOM参数项：
- 采样频率（Hz）；
- 深度层数 K;
- 计算窗口长度W（秒）；
- 关键阈值：imbalance偏多偏空阈值（默认0.6/0.4）、z-score判定阈值（默认1和1.5作为一般和强信号）、吸收判定阈值（如累计次数2次补单交替，或参数α=0.6）等;
- 策略参数：延迟入场秒数、止损和止盈tick预设值等;
- 预设配置下拉框：例如提供 “保守 / 标准 / 激进” 三种预设，一键填入不同阈值组合 [65] [66]；
- **保存** 按钮：校验输入后，将参数通过 API 保存（后端会持久化，如写入配置文件或数据库）并更新本地状态 [67]。这些参数也可以同时保存到浏览器的localStorage方便下次加载 [67]。
- **启动/停止控制**：在DOM标签页内，提供 **“启动 ▶ / 停止 ■”** 按钮，用于开启或关闭DOM数据监控 [67]。启动时，前端调用后端 API（`POST /api/monitor/start`）开始订阅数据 [68]，并建立 WebSocket 连接接收推送。停止时，调用 `POST /api/monitor/stop` 终止订阅 [68]。启动后按钮状态切换为“停止”可用。
- **数据展示刷新**：当监控启动后，DOM标签页中的指标图表应该开始实时刷新。为平衡性能，前端绘图帧率建议限制在 5~10FPS [69]。如果DOM标签未处于激活状态（用户没看此页），可以暂停绘图更新以节约资源 [69]。
- **导出功能**：提供**导出CSV**按钮，允许用户将当前收集的DOM告警事件列表导出 [70] （或整个近期数据导出），便于离线分析。这可调用后端数据查询接口，将结果另存为CSV文件。
- **界面样式**：遵循当前主题（支持暗色/亮色模式切换）。可利用CSS变量实现一键换肤 [69]。确保新元素（图表、表格、按钮）与整体UI风格一致，比如使用相似的配色和字体。
- **兼容性**：如果DOM功能未开启或无数据，标签页可显示“未启动DOM监控”提示。当切换账号或合约时，需刷新内容为对应symbol的数据。

2. 实时监控弹窗增强：很多交易系统有针对单一合约的“监控弹窗”（包含价格、指标、小图表等)。本设计将DOM分析结果集成到该弹窗的图表区域上方，以提供**即时的做多/做空建议可视化**：

- 在弹窗的行情图表上方，新增一个横向的**信号条区域**（宽度与图表相同，高度适当，比如图表高度的20%）。此区域分为上下两部分：上半部分表示**做空信号强度**，下半部分表示**做多信号强度**。默认背景为中性透明，中间可有一条中线分隔。
- **信号强度可视化**：采用颜色深度体现信号强弱——**红色**表示偏空信号，**绿色**表示偏多信号。具体实现：每当有新数据帧（每秒）到来，计算一个综合的做空推荐强度和做多推荐强度。可简单地使用 `trend_score` 作为基础（负则空，正则多）或者根据满足信号条件的数量来评分。然后在上半条对应

时间位置绘制一个透明度或饱和度与空头强度成比例的红色矩形，在下半条绘制与多头强度成比例的绿色矩形。随着时间推进，这些矩形形成连续的色条，用户一眼可看到哪个时刻多/空信号更强。

- **时间同步**：该信号条与下方价格K线图时间轴同步滚动。例如图表显示过去5分钟行情，则信号条也显示过去5分钟的DOM信号情况。新信号矩形从右侧出现，旧的向左滑动。如果图表支持缩放，信号条应相应缩放聚合（例如缩短窗口则合并信号）。
- **交互提示**：用户将鼠标悬停在信号条某一时刻，会出现一个tooltip/浮层，显示该时刻详细的DOM分析结果，包括：当时的关键事件和建议操作。具体：
- **事件列表**：列出此刻检测到的事件标志，如“sweep_down”（向下扫单）、“z_pull_bid 高”（买盘撤单强烈）、“absorb_up”（上方吸收）等。通过这些术语提示用户盘口发生了什么情况。
- **推荐操作**：基于事件给出的建议措施，如“延迟入场”、“减仓”、“收紧止盈（TP）”、“不追多/空”、“平半仓”等 [71]。这些建议对应前述策略规则：例如出现 `pulling_buy`（买盘大撤单）则建议“延迟入场”多单；出现上方吸收则建议“多单减仓或提前止盈”；出现剧烈扫单则对于反方向仓位建议“立即止损/平仓”等。tooltip里可以呈现类似**“事件: 卖盘扫单; 建议: 切勿追多”**的信息，帮助交易者当下决策。
- **结合价格图表**：由于DOM信号偏短线，而监控弹窗的K线可能是5分钟甚至更长周期，为了更直观，可以**切换或添加高分辨率图表**模式。当DOM监控激活时，允许用户将图表切换到例如**1秒线**或**tick图**模式。这使得价格的微观波动与DOM信号能够一一对应查看。如果不切换，也可考虑在原K线图上标注出DOM信号事件的位置，例如在发生吸收或扫单时，在K线上加特殊标记（比如三角或圆点标记，并用颜色区分事件类型）。但标记过多会杂乱，因此**推荐**的方法是：**在DOM活跃时，引入小周期图表**同步更新，让用户观察细节。例如可以在弹窗增加一个子图：上方为常规5分钟K线概览，下方为最近几分钟的秒级别价格走势叠加DOM事件标记。
- **性能**：因为弹窗叠加了新的动态绘制元素，需要注意性能优化：采用Canvas或SVG绘图，预先创建固定大小元素，只更新属性来移动显示。绘制频率可以和WebSocket更新频率一致（例如5Hz），不要超过必要频率。对于hover提示，可以在前端缓存最近几秒的数据以快速访问，不每次都请求后端。
- **可选隐藏**：提供选项让用户自行选择是否显示DOM信号条（比如在弹窗设置里增加“显示DOM建议”勾选项）。部分用户在不需要时可关闭以简化界面。

3. 通知与告警界面：DOM模块的重要作用之一是及时告知用户异常情况，因此需要和通知系统结合：

- **事件通知**：对于超出阈值的关键事件，系统应发送通知。用户可在**Notification Settings**中针对DOM事件选择通知方式（如App内弹窗、声音提醒，或通过Telegram/邮件推送）。
  例如，当 `fake_breakout_prob` 超过某阈值（如0.8）或出现 `sweep_down` 事件时，立即触发警报通知：“DOM警报: 检测到向下扫单，可能出现快速下跌风险”。如果用户配置了Telegram通知，后端将调用TelegramNotifier发送消息 [72] [73]。通知内容应简洁明了，包括合约、事件、时间和建议操作。
- **风控警报**：当DOM策略执行了某些风控动作时（如**动态减仓**或**停止交易**断路器触发），也需要提示。可复用现有的S Rules告警机制，将这些情况记录为一条风险告警。例如：“策略已将仓位缩减50%（吸收卖盘信号）”或者“DOM策略触发连亏停止，已暂停交易”。这些消息可以出现在S Rules告警列表中，Level设为warning或critical，以引起注意 [20]。如果S Rules告警格式不便于承载DOM策略信息，也可新增“DOM Alerts”列表，但是为了统一，优先尝试集成。
- **前端展示**：前端应能显示上述通知。在应用内，可能有通知中心或弹窗。对于DOM事件，可以设置**不同优先级**：如一般信息类事件（imbalance普通变化）不提示，重大事件（sweep、假突破）立即弹窗。并提供**ACK**按钮让用户确认关闭警报 [74]。在DOM单页模式下，也有告警表格列出历史事件 [75]。在集成UI下，可以考虑点击DOM标签页或弹窗的某个区域打开最近DOM事件列表。
- **避免干扰**：通知频率需受控，避免每秒弹多个。可以对类似事件设最小间隔，或者在一定时间窗口内合并消息。例如若连续多秒检测到吸收，可每30秒提醒一次“持续出现吸收行为…”。

4. 独立DOM监控页面（备选方案）：最初在设计文档中提出过一个将DOM监控放在单页的方案 [76]。其布局包括左侧参数控制栏、上方价格及指标图表、下方深度热力图及告警/建议列表 [77]。这种模式下所有DOM信息集中一屏，交互直观。我们可以将其作为一个**高级/详细视图**提供：

- 用户通过点击某个按钮（如DOM标签页中的“独立视图”链接）进入单页模式。在该页面，可以看到更加详尽的可视化：例如**盘口深度梯形图/热力图**（直观显示买卖盘在各价位的挂单量分布，颜色表示量大小），每秒更新1-2次 [69]。同时有**告警表格** [75] 列出所有事件，支持按类别/方向过滤 [78]、点击展开详情（比如前后10秒深度情况） [74]。底部还有**推荐操作按钮栏**，用户可一键执行建议动作（如点击“延迟入场”会通知策略不下单或推迟） [79]。
- 由于单页模式重复了部分功能（例如参数设置、启动停止等），需要确保其操作会同步影响集成模式（例如在单页调整参数，保存后也作用于全局DOM设置）。这一页面可以共用后台WebSocket和API逻辑。
- 单页模式更适合专业用户深入研究盘口，或在多屏幕环境下专门监控。而平时大多数用户可能只需要集成在监控弹窗内的精简视图。因此我们的UI设计以**集成为主、独立为辅**。开发时可以先实现集成部分，再根据需要实现独立页面（代码上很多可重用，例如绘图和参数组件）。

总之，前端UI会提供**从参数配置 -> 实时监控 -> 告警通知 -> 策略执行**的全流程可视化闭环。交互上注重**及时反馈**和**易读易懂**：用颜色和简短文案提示风险，用简洁按钮让用户快速响应，同时在需要时也能展现详细数据供分析。

后端设计

后端将基于现有 **FastAPI** 框架扩展，实现DOM数据处理、传输以及与策略和通知系统的集成。设计要点如下：

数据处理流程：
- **数据采集异步任务**：启动DOM监控时，后端应创建一个专门的异步任务或线程与IB API通信，订阅指定合约的Market Depth和Tick数据。由于IB API可能是同步回调模式，可以使用线程将数据放入 asyncio 队列，再由AsyncIO任务处理。确保**线程安全**和**不阻塞**主应用。
- **盘口快照组装**：每次从IB接收到深度更新或成交，更新当前盘口状态（买卖档位价格和累计量）。可以设置每100ms检查一次是否有新变化，并生成快照（或者每来一条数据就更新最新帧并立即推送，实际效果接近高频连续）。注意IB深度数据需要维护一个当前Order Book状态（逐条update vs完整快照模式），实现上维护两个长度K的数组表示当前买一到买K价和量，同理卖侧，每次更新相应位置即可。
- **计算引擎**：在生成每个新快照后，计算上述所有指标：imbalance直接算；depth Δ由当前与前一帧比较；CVD通过滚动累加成交计算；OFI根据当前best价与之前best挂量差计算 [24]。实现细节：使用NumPy对环形缓冲批量计算z-score。也可维护滑动平均和标准差来即时标准化（例如采用W秒窗口数组，计算均值和方差）。对于事件检测的复杂条件，可直接在python判断当前帧的指标是否满足阈值。
- **结果存储**：将当前帧的关键信息放入适当的数据结构（比如一个dict)并写入**事件列表**或**指标时间序列**。对于短期内推送的数据，可以保存在内存里的缓冲；对于持久化需要的，则在后台异步批量写入磁盘。特别地，**告警事件**（absorb/sweep等）和**策略触发**要记录下来（内存列表或数据库），供查询和UI显示。

WebSocket 通信：
- FastAPI集成WebSocket用于推送实时数据。可以增设 `/ws/dom/{symbol}` 端点，前端连接后，后台任务每当有新快照计算完毕，就通过此WS发送消息。消息格式为 JSON，比如：

```
{
  "ts": 1631272025000,
  "imb": 0.67, "z_stack_bid": 1.2, "z_pull_ask": 0.0,
  "z_cvd": 1.1, "z_ofi": 0.5,
  "fake_break_prob": 0.2, "trend_score": 0.8,
  "events": ["sweep_down"],
```

```
  "suggestion": "delay_entry"
}
```

其中 `events` 列出当前帧触发的事件名称（如果有），`suggestion` 给出主要的建议动作编码（如果有明确建议）。前端据此更新图形和tooltip显示。 - 需要考虑**流控**：如果行情过于频繁导致计算帧率高于前端处理能力，我们可以对WS发送频率限流（例如最多10Hz） [69] 。或者如doc中所示监控 `ws_backpressure` 等指标 [80] 来动态调整。 - 当用户停止DOM监控或离开页面时，应关闭WebSocket。后端也应在适当时候（比如stop调用）关闭相应连接，避免后台推送无人在监听。

REST API 接口：
- **启动/停止**：提供 `POST /api/dom/start` 和 `POST /api/dom/stop`（或整合进 `/api/monitor/start?type=dom`）来控制订阅。启动接口接收参数如 symbol（和可能的配置参数覆盖）。成功启动则返回确认以及WS连接信息；停止则终止后台任务订阅。
- **参数配置**：提供 `GET /api/config/dom` 获取当前DOM参数设置，以及 `PUT /api/config/dom` 更新设置 [67] 。参数包括K, W, 各阈值等。配置可存在全局配置文件或数据库，也可以按用户个性化（若系统支持多用户并各自配置）。这里简单假设单用户，存在一个JSON配置文件。 - **数据查询**：`GET /api/dom/data`，支持查询历史DOM指标序列。参数如symbol、起止时间等 [20] 。返回JSON数组或CSV：每元素含时间和各指标值。若数据量很大，也可要求加limit或分段获取。对于UI上的小图，可以默认查询最近X分钟数据。 - **告警获取**：`GET /api/alerts?type=dom` 返回DOM模块产生的未确认告警列表 [70] 。结构包括时间、级别（info/warn/critical）、事件类型、描述、建议等。前端用于在表格中显示 [20] 。
`POST /api/alerts/{id}/ack` 标记某条告警为已读 [74] 。 - **执行建议**：如果保留手动执行推荐操作的模式，提供 `POST /api/recommendations/execute` 接口 [74] 。参数包含建议类型（比如"delay_entry"）及适用对象（symbol或order）。后端收到后，如果是策略相关操作，可以记录日志或实际执行（如暂停某策略一段时间）。在自动交易模式下，此接口主要用于UI记录或人工确认逻辑。 - **策略开关**：可提供 `POST /api/strategies/DOM/enable` 之类的接口，用于启用/停用DOM自动交易策略。或者复用已有策略管理接口，通过策略ID来更新状态。 - **策略信息**：为前端策略面板提供DOM策略的详细信息，可扩展 `/api/strategies/{name}/internal-info` 返回该策略特有的信息 [81] 。比如添加DOM策略当前参数、最近信号等，丰富Summary页内容。

自动交易执行：
- **订单路由**：DOM策略下单可以直接调用现有订单管理模块（如通过OrderManager或IB API接口）。需确保订单带上策略标识。可能在下单函数中增加参数 `strategy="DOM"`，数据库中存储订单时记下。
- **止损/止盈执行**：可以利用IB的**Bracket Order**功能发送附带止损止盈的单子。如果未用Bracket，则需要在策略里监视价格以手动发送止损单。由于DOM策略会管理很多细节，使用Bracket减少后续管理复杂度。
- **动态调整**：如移动止损到BE，可以通过监听价格或在达到目标后发出修改止损指令。这逻辑可以写在策略内部，或借助风险规则模块实现。
- **冲突处理**：若用户手动下单或其他策略也交易同一合约，需要约定优先级或互斥。简单处理是限制DOM策略只在它独占的环境运行（比如不同帐户或合约，或用户明确知道不要手动干预DOM策略管理的仓位）。本设计暂不深入策略协调，但应在文档中提醒用户。

通知模块：
- 系统已经有 TelegramNotifier 等实现 [72] 。DOM模块需要在合适的地方调用这些通知方法。比如当 `fake_breakout_prob` 超过0.8并且auto交易仍持仓，可以调用 `TelegramNotifier.send_message("Fake breakout alert for MNQ...")` 等。这些调用可以放在事件触发检测的代码分支中。由于TelegramNotifier使用async方法 [73] ，确保在FastAPI（asyncio）上下文中正确await。 - 还可以实现本地WebSocket通知或浏览器通知（HTML5 Notification）。对于浏览器，我们可以在WS消息中加入alert字段，前端JS监听到重要alert时弹窗声音提示等。 - 通知配置通过环境变量或配置文件（Telegram的token/chatid在dotenv里配置） [72] 。后端应在启动时初始化notifier，如doc代码所示，如果没配置则warning提示 [82] 。

数据模型与格式：
- **DOM Snapshot**：内部使用NumPy，但对外API/WS需序列化。可定义Pydantic模型例如:

```
class DomSnapshot(BaseModel):
    ts: datetime
    bid_depth: int
    ask_depth: int
    imb: float
    z_stack_bid: float
    z_pull_ask: float
    z_cvd: float
    z_ofi: float
    events: List[str] = []
    trend_score: float
    fake_break_prob: float
    # ...等
```

事件列表可以是简单字符串数组，也可进一步分类结构，但字符串已足够。 - **Alert**：如前述，包含时间、级别、类型、消息、建议字段 [20] 。存储上可以简化为日志表，每条记录有自增ID。已确认的alert可以打标记不再返回。 - **存储格式**：半持久化数据建议Parquet，可通过pandas DataFrame写。Parquet优点是压缩高效、列存利于取部分字段（例如查询imbalance列）。考虑到一周数据也不算太大（几十MB），也可简单存CSV文件按天，读取时全部读入pandas筛选。 - **性能与资源**：
- 内存：环形缓冲约10000帧 * 2 sides * 30 levels * 2 fields * 4 bytes ≈ 4.8百万字节 (~4.8MB)，可接受。多个symbol并行则按比例增。
- CPU：指标计算主要是向量减法、加法和少量条件判断，10Hz频率Python完全能应付；若性能不足，可用Numba加速或C扩展，不过预计不需要。
- IB限制：需注意IB对深度数据请求有速率限制（典型100ms发一次增量），本模块10-20Hz其实就接近IB频率上限，应该不过度消耗请求额度。Tick-by-tick成交订阅也要留意并发数量上限。 - 日志：在调试阶段，可打印每次信号判断结果（imbalance, signals等）方便验证。但上线后应减少日志量，仅在发生交易或异常时记录。

与现有逻辑关系：
- 由于此模块新增大量功能，需要确保与原系统解耦良好。例如，不强依赖原有风险规则模块的实现（因为已有的不成熟，需要重新确认实现方式）。可以在设计上将DOM模块相对独立：独立的数据处理线程，独立的策略模块，不修改原有策略代码，仅通过公共接口（如下单接口、通知接口）交互。
- 原有UI和backend可能有部分类似功能（例如风险告警)。我们要复用则复用，复用不了就parallel实现。例如现有策略模板/事件订阅机制不可靠，就为DOM策略定制实现，不在别的策略上通用，以降低干扰风险。
- 数据层面，如果已有数据库，需要考虑是否将DOM数据存入其中。若没有，就以文件为主保存。此外DOM参数可以放在系统已有的config结构中（比如数据库配置表）。 - 对用户来说，除了UI上多了DOM页和可能多了策略，其他操作不变。保持用户体验的一致性：比如依然可以在策略面板里启停策略，依然从通知中心看告警，没有引入复杂的新操作流程。

测试与验证

为确保DOM分析模块可靠有效，需要在各个阶段设计充分的测试：

1. 数据及时性和完整性测试：
- **频率验证**：模拟一个高速行情环境（或直接在交易时段测试），在启动DOM订阅后，记录每秒实际接收到的帧数。期望值是接近配置的10Hz。如果显著低于（比如只有1-2Hz），需检查是否IB限制或代码瓶颈。反之，若因为行情太活跃导致内部计算跟不上，也要发出警告。可以在模块内维护计数器，比如每秒计算帧数，输出

`ui_fps` 等 80 对比目标。
- **丢帧/乱序**：通过在Debug日志中记录每帧时间戳差，确保没有异常大的间隔和没有倒序。如发现空洞（时间跳过很多毫秒无更新）或乱序，则检查线程/队列同步问题。
- **极端情况**：测试在开盘瞬间、数据发布等行情喷发时，模块是否还能处理。比如用历史数据重放给模块看是否缓冲环可以装下（若>WINDOW帧在短时间涌入则超出，不过按当前WINDOW设计足够几分钟高频，不会秒内就爆）。另外测试停止订阅再重新启动，数据是否干净重置，旧数据是否清除等。

2. 指标计算准确性测试：
- **单元测试**：针对每个计算函数编写单元测试。构造小的DOM数据序列（可手工）来验证结果。如：
- imbalance：构造买卖盘量比如买=100, 卖=50，计算是否=0.667匹配 21 。
- z-score：构造一段已知均值和std的数据，看计算是否正确，尤其滚动实现要对齐pandas结果。
- OFI：手工计算几步best_bid_sz变化和中位价方向，对比函数输出。OFI较易出错边界是当价不变或同时变，这里确保 `1_{mid_t >= mid_{t-1}}` 逻辑准确。
- Absorption事件：制作一个序列，其中连续5秒每秒都有buy_vol很高但价格未涨，同时ask挂单每秒被减又补，这应该触发absorb_up，看模块flag是否=True。
- Sweep事件：类似构造2秒内bid挂单减少某阈值且sell_vol暴增，检查sweep_down触发。

- **模拟实盘数据测试**：录取一段真实行情DOM数据，离线计算指标（例如用Python/Pandas按照设计公式算出imb, z值, 事件），然后运行模块对同样数据流，看输出是否一致。尤其是**拐点**：当imbalance正好过0.60那刻，模块应该在那帧输出事件或不同信号。通过与离线结果比对，验证计算正确性和无时序偏差（例如滚动窗口末尾对齐问题）。

- **策略规则测试**：通过编写**场景测试**来验证信号判定：

- **做多信号场景**：喂给模块一个下跌到支撑并出现挂单大增、成交跟进的模拟数据，检查模块是否在预期时刻输出“long_signal=True”。改变其中一个条件（如没有成交跟进）看信号是否按规则不触发。
- **做空信号场景**：类似对压力位测试。也测试硬性规则：当价格不在支撑/阻力范围时，即使imbalance极端也不触发信号，看模块能否抑制。

- **连续条件**：测试多种事件组合在短时间发生时，fake_breakout_prob的值变化是否合理。例如手动标记某帧满足假突破条件，看看模块输出prob有没有接近1。

- **输出一致性**：检查模块输出给前端的数据是否自洽。例如一旦有 `sweep_down` 事件输出，对应的z_pull_bid和sell_vol数据也应高于阈值，否则表示逻辑不一致。通过随机抽取一些输出帧，人工审查它们的各字段逻辑正确性。

3. 前端交互测试：
- **UI功能**：手动测试DOM标签页和监控弹窗：
- 进入DOM页，输入各种参数（包括边界值，如K=1或K=30、阈值0或极大），测试即时校验是否正确（如无效输入是否阻止保存）。保存后刷新页面看看配置持久化是否生效。
- 点击启动，确认WebSocket连接建立（Network面板查看）。DOM图表开始更新，应看到曲线在动或数值变化。
- 切换离开DOM页再回来，图表是否继续更新或暂停。
- 停止后，图表应停止且按钮状态正确。尝试重新启动，是否能重新出现数据。
- **信号条**：在监控弹窗中观察信号条颜色变化是否符合预期：可以选一段实际行情，如果在真实支撑位看到大买单，应该对应底部绿色条变深。用模拟或历史回放造景也可以，比如让imbalance持续高，看是否绿色持续高亮。
- Hover提示：将鼠标移到红/绿条明显的地方，看tooltip内容。验证列出的事件和建议与后台实际发送的数据一致（可以在后台打印或者借助浏览器调试看ws消息）。检查中文文案是否清晰、没有错别字，建议语句是否合理（比如不会出现空白建议或错乱组合）。对不同组合事件的tooltip都过一遍。
- 时序对应：当tooltip出现时，对比下方价格K线，确认对应的时间对上。例如信号条上标记了10:05:30有吸

收，K线应是在10:05左右的一根。若错位需要调整同步逻辑。
- **性能**：人工快速拖动时间轴或缩放，看信号条是否同步变化，没有明显卡顿或错乱。在极端情况下（例如1秒内10个事件瞬发），前端能否处理渲染，若不行可能需在后端降频，所以也在测试中观察CPU和内存使用。

4. 通知与告警测试：
- 配置Telegram测试：填入自己的测试群ID和Bot Token [72]，触发一个事件（可以暂时在代码中强制触发通知函数），查看Telegram消息是否收到，格式是否正确（如符号emoji、换行）。尝试触发多条，看是否每条都能发送或是否有Rate Limit问题（Telegram有每秒30条限制，正常不会超）。
- 应用内告警：故意制造一个触发断路器的情况（比如模拟3笔亏损），看S Rules列表是否新增一条停止交易的警报。如果有就ACK它，看能否在界面消除 [74]。
- 重复事件合并：测试模块在持续满足某条件时不会疯狂发通知。例如持续高imbalance 5分钟，不应每秒都通知“imbalance>0.6”，这点在实现时可能已有防抖措施。测试时可以打开日志确认这一逻辑有效。
- 交互：点击通知链接（如果通知包含点击跳转，比如点击打开DOM页）是否正常；在各种通知触发时，系统稳定性（不断弹通知会不会拖慢前端或导致崩溃）。

5. 策略交易测试：
- **模拟账户**：在仿真交易环境或非常小仓位下测试DOM自动交易逻辑：
- **入场正确性**：在接近设定支撑时触发做多信号，看策略是否真正下多单。检查下单数量是否等于按照公式计算的（测试时可固定Eq和r计算出N0f）。止损单是否同时挂出在正确价位。目标价若实现是否平仓。
- 空仓等待：当没有满足3条件的信号时，即使价格在结构位也不应下单。测试在2条件满足的情况下策略不交易，3条件才交易。
- 延迟与过滤：模拟一个应该延迟的情境：比如满足做多条件瞬间又出现pulling_buy事件。策略应检测到，执行“延迟入场”。可能实现为延迟若干秒再判断条件是否依然成立再下单，或直接跳过这次信号。测试这种情况下，策略确实没有立刻下单，而是等一会或者放弃。如果pulling信号过去后条件仍满足再下，则验证延迟机制运作。
- 减仓：对于已经持仓的情况，模拟出现absorb信号。根据设计，策略可能平掉一半仓位或收紧止盈。测试看有没有相应行为：例如absorb_up出现后，策略发出部分平仓指令（可以通过日志或订单记录核实）。
- 停止交易：强制让策略连败3次（这可能通过历史回测或手动干预让它连续亏小单）。检查第四次信号出现时，策略不应下单。日志或界面应提示“停止交易，断路器已触发”。然后尝试手动恢复（新的一天或重置参数）前才允许交易。
- PnL统计：让DOM策略运行一段时间，之后在策略面板查看其Orders & PnL：每笔交易方向、盈亏是否合理，累计PnL是否正确。特别关注部分平仓是否计入正确（如平半仓应该算一次开仓一次平仓两条记录，统计时不重复计算）。
- 综合效果：如果有条件，将模块在仿真环境跑几天，对比开启DOM策略和不开启的收益曲线和回撤。预期该策略不会显著盈利但能避免一些陷阱亏损*。例如统计假突破次数以及策略是否成功避免亏损或小亏代替大亏。根据这些结果再调整算法参数。

6. 可靠性与安全测试：
- **长时间运行**：让模块持续运行>8小时，观察是否有资源泄漏（内存应稳定在某范围，文件句柄无泄漏，CPU无渐增）。尤其检查websocket是否稳定（不该长时间断开），IB连通在长时间后是否保持（IB有时需要ping，若断线模块要能重连）。
- **异常处理**：模拟IB连接中断（停止Gateway，或拔网线），看模块是否及时检测并通知（可能触发连接警报通知) [83]。恢复连接后是否自动重订阅或需要手动启动。确保断线期间不崩溃并正确清理资源。
- **并发**：尝试同时对不同符号启动DOM监控（如果允许多并发的话），看看是否互相影响。如果资源不支持多并发，则UI需要限制一次只能监控一个。此测试确保不会因为用户误操作导致线程冲突。
- **数据库一致性**：如果涉及数据库记录订单/告警等，测试在异常终止时（比如程序崩溃）不会丢失已执行订单的记录，或下次启动能够恢复正确状态（如已触发的断路器不丢失等）。
- **安全**：确保API权限控制，没有未授权即可订阅他人数据或发送交易的漏洞。比如 `/api/dom/start` 应该需要登录态。测试未登录调用被拒绝。
- **回退方案**：在测试中预想如果DOM模块不工作（比如因为IB数据问题或算法bug导致错误信号），不会把系统

其他部分拖入危险。通过模拟故障观察，例如让模块误发极端信号，确认不会自动下超出范围的单（因为仓位算法限制N最低1手且f<=1，不会无限加仓或反向开仓）。

通过上述详尽的测试和逐步改进，我们能确信模块的**数据正确、信号可靠、交易安全**，最终在实盘中发挥预期作用。

项目实施计划

鉴于项目复杂度高，但开发人手仅“一人+AI”，我们将分阶段循序渐进完成，每步都有明确产出，方便AI逐步实现代码和文档：

1. **需求确认与设计细化**
2. **输出**：本设计方案文档定稿，并经确认无遗漏需求。核实DOM指标定义与行业惯例一致性（根据文档和资料校验公式准确性 84 85 ）。确定关键支撑/阻力的获取方式（假定由用户手工标注或简单配置）。
3. **数据采集模块开发**
4. **任务**：编写IB API连接和数据订阅代码。封装一个 DepthSubscriber 类，内部维护当前订单簿状态。实现 `start_depth_stream(symbol)` 和 `stop_depth_stream()` 接口，以及在新数据回调时调用计算模块。
5. **测试**：在测试账户上订阅一个品种深度，打印输出前几条深度更新和tick数据，验证连接通畅、数据格式正确。
6. **环形缓冲与核心计算实现**
7. **任务**：实现NumPy环形缓冲数据结构，提供 `append_snapshot(book_state)` 方法自动处理索引循环。编写计算函数 `compute_features(snapshot_window)`：传入最近W秒的数据窗口（或最近N帧），返回当前imbalance、各z-score、CVD、OFI等值。实现事件判定函数 `detect_events(features_history)`：根据当前及短期历史的特征序列判断absorb/sweep。整理信号判定规则函数 `check_trade_signal(features, price_structures)` 返回做多/做空信号布尔。
8. **测试**：为计算函数设计单元测试（用预设矩阵数据验证imbalance、z-score、CVD计算准确），为事件函数设计断言测试（构造场景触发absorb看返回flag）。
9. **后端服务接口开发**
10. **任务**：使用FastAPI创建路由： `/api/dom/start` (启动监控), `/api/dom/stop` (停止监控), `/api/config/dom` (GET/PUT参数), `/api/dom/data` (GET历史数据), `/api/alerts` (GET/ACK告警) 等。实现 WebSocket `/ws/dom/{symbol}`。这些接口函数内部调用第2步的数据订阅模块和第3步的计算模块。确保在启动时启动后台任务（可用 `background_tasks` 或自建async task)处理循环读取和推送WS。
11. **测试**：用FastAPI的docs界面或curl测试启动停止接口，观察后台日志确认线程启动/终止。测试配置GET/PUT是否读取更新正确。写一个临时前端html用JS连接ws，打印接收的数据，检查数据格式和字段合理性。
12. **前端DOM标签页实现**
13. **任务**：修改前端页面结构，新增DOM选项卡和内容DIV。使用HTML/CSS搭建指标显示区域和DOM Settings对话框。利用JS实现参数表单的打开/关闭、数据校验、保存(fetch调用后端PUT)。绘制简单折

线图或数值显示，初期可以用Canvas API或简单SVG。实现“启动/停止”按钮事件，调用后端API并根据返回结果控制WebSocket连接开启关闭。处理WebSocket消息，实时更新图表数据。

14. **测试**：在浏览器中打开修改后的页面，切换到DOM页，尝试调整参数保存，看Network请求正确。点击启动后，看Console中WebSocket是否收到消息。人为构造fake数据（或借真实小数据）喂入，验证图表呈现。

15. **前端监控弹窗整合**

16. **任务**：在监控弹窗组件中集成DOM信号条显示。由于可能已有chart库，需要判断是集成在现有chart上层绘制，还是增设一个canvas层。实现绘制上下双色条，根据接收的WS数据更新条的位置和颜色。实现tooltip：可以用一个隐藏的div，鼠标hover时定位并填入内容。编写逻辑将WS来的事件列表转成人类可读文字（可做个映射字典）。

17. **测试**：启动DOM监控，对比弹窗信号条色彩随行情变化。手动模拟若干事件组合，触发tooltip显示，检查内容准确。尝试关闭DOM显示选项（如果有），看信号条隐藏。

18. **通知与告警功能实现**

19. **任务**：在后端计算逻辑中，加入当某事件触发且满足通知条件时，调用通知函数。实现一个简单的 `Notifier` 类，可以封装TelegramNotifier或其他。确保使用异步发送而不阻塞主流程。实现告警存储，将需要记录的事件插入Alert列表/数据库。后台提供task定期清理过期告警或限制列表大小。前端上，在已有通知中心集成DOM告警显示（也许无需改很多，如果已有通用通知弹窗，只需在server发时指定类型）。

20. **测试**：配置一个低门槛，例如imbalance>0.5就通知，触发一次看看系统行为。观察日志中send_message被调用并返回成功。查看前端通知UI是否出现消息，以及alert API返回是否包含新记录。然后ACK之，通过API或UI，看记录是否标记。

21. **自动交易策略实现**

22. **任务**：开发DOMStrategy类，继承自系统策略基类（如StrategyBase）。实现其启动方法（可能就是订阅DOM信号事件），以及处理信号的回调。在信号触发时执行下单：构造Order对象，计算数量和价格（市价或挂当前中间价±滑点）。利用系统订单管理API提交。实现平仓/止损：可以直接使用risk_rules中的止损规则，或在策略中跟踪订单状态。当触发断路器时，在策略内部设置一个flag阻止后续交易，并推送一个告警。最后，将策略注册到策略管理器，使其在启动程序时自动加载。

23. **测试**：在模拟环境运行策略：提供假的DOM信号触发函数调用，检查策略是否按照预期下单（可以mock OrderManager，捕获订单参数验证）。测试断路器逻辑：手动调用策略的亏损计数3次，看flag是否阻止第4次下单。测试启停：关闭策略或禁用开关，看就算有信号回调也不下单。

24. **策略面板和报告**

25. **任务**：确保前端策略面板能识别DOM策略。可能需要在策略列表硬编码加上DOM策略（因为默认就有）。若internal_strategy_layout已有支持，只要策略标识为internal类型，就会自动有Summary/Orders/PnL标签 60 。撰写DOM策略描述，显示在Summary。修改internal-info接口，让DOM策略返回risk_rules字段（比如默认止损%或断路器参数） 86 。实现PnL统计，如果系统已有全局计算，只要订单有策略ID即可。若没有，则可能写简单逻辑汇总。

26. **测试**：启动前端，打开策略面板，看是否出现DOM策略条目。点进去Summary看描述正确，Orders页应显示之前测试下的单，盈亏统计与订单对应（若策略无历史则为空）。确认API拉取internal-info包含我们期望的数据格式。

27. **综合联调测试**

    - **任务**：将前后端连接一起来一遍“端到端”测试。在测试环境接入实时行情或回放历史数据：
    - 打开DOM页面，启动监控，看前端曲线和弹窗信号是否与实际盘口状况相符。
    - 在一个模拟账户上让DOM策略跑，看看实际下单表现。如可能选一个较慢的品种以避免风险。观察几笔交易结果是否符合我们的信号规则期待。
    - 整体测试过程中，记录发现的问题（比如阈值不合适导致过多/过少信号，UI某处数值不更新等），整理修正。

28. **参数调优与迭代**

    - **任务**：基于测试结果调整参数或逻辑。如发现某阈值过低频繁误报，则提升一些，或者引入动态调整（比如根据不同时段自动切换参数） [87]。如果发现新的模式没捕捉到，可考虑增加指标或规则。在初始版本稳定后，也计划后续迭代如doc提到的优化点 [87]：
    - 区分日盘(RTH)与夜盘(ETH)的均值和σ，因为流动性差异大，阈值可分时段校准 [88]。
    - 根据历史交易结果每周自动优化阈值（网格搜索或其他统计方法），不断提高信号质量 [89]。
    - 增加监控对错误信号的捕捉：如imbalance高却CVD下行（误多信号），或sweep_down后被迅速拉回（误空信号）等情况，识别出来下调这些条件的权重 [90]。
    - **测试**：每次调整再跑回测或沙盒观察改进效果。

29. **上线准备**

    - **任务**：整理用户文档和风险提示。将模块配置项、使用说明添加到系统手册中。特别地，教育用户如何解读DOM信号条和tooltip内容，以及在自动模式下需要注意什么（如不要同时人工反向操作等）。
    - 部署前进行代码Review，确保日志级别、错误处理都恰当。然后将新模块部署到生产环境（注意在非交易时段部署）。实时观察首日运行，确保没有报错。

整个实施过程中，开发者可以充分借助AI编程助手来生成和改进代码片段，加速进度。然而，每一步都务必人工分析验证AI生成的逻辑，与本设计预期一致，再进行下一步。通过以上阶段划分，我们能够稳步完成模块开发，并最终交付一个健壮的DOM数据分析系统。

结论

本方案详细规划了一个**基于DOM盘口数据的交易辅助分析模块**，涵盖从背景原理、算法设计到前后端实现和测试部署的全流程。通过将微观盘口信息与宏观价格结构相结合，系统可以识别出许多常规技术分析难以察觉的异动（如大单压盘/撤单、虚假突破等），并将其转化为**具体的风控建议和交易动作**。整个模块将无缝融入现有交易平台：后端以高速缓冲和异步计算确保性能，前端以直观可视化和交互降低使用门槛，通知和策略联动则实现了从**发现 -> 提示 -> 执行**的一体化流程。这不仅有助于**过滤噪音、避免陷阱**，也可以在关键时刻**确认趋势、优化进出场**，从而提升策略收益稳健性。通过严格的测试和逐步优化，我们有信心该模块能够在实战中发挥效用，成为交易者的重要利器。

---

[1] [2] [3] [4] [21] [22] [23] [24] [25] [26] [27] [30] [31] [32] [33] [36] [37] [38] [39] [40] [41] [42] [43] [44] [45] [46] [47] [48] [49] [50] [51] [52] [53] [54] [59] [85] [87] [88] [89] [90] mnq_dom_design2.md
https://github.com/winglight/ibbot/blob/d0e784fc22e6f1cc03cbd3419e37546dfd8b5c43/docs/mnq_dom_design2.md

[5] [6] [7] [8] [9] [10] [11] [12] [13] [14] [15] [16] [17] [18] [19] [20] [28] [29] [34] [35] [55] [56] [57] [58] [63] [64] [65] [66] [67] [68] [69] [70] [71] [74] [75] [76] [77] [78] [79] [80] [84] mnq_dom_design.md
https://github.com/winglight/ibbot/blob/d0e784fc22e6f1cc03cbd3419e37546dfd8b5c43/docs/mnq_dom_design.md

60 61 62 81 86 internal_strategy_layout.md

https://github.com/winglight/ibbot/blob/d0e784fc22e6f1cc03cbd3419e37546dfd8b5c43/docs/internal_strategy_layout.md

72 73 82 83 telegram_notifier.py

https://github.com/winglight/ibbot/blob/d0e784fc22e6f1cc03cbd3419e37546dfd8b5c43/src/telegram_notifier.py